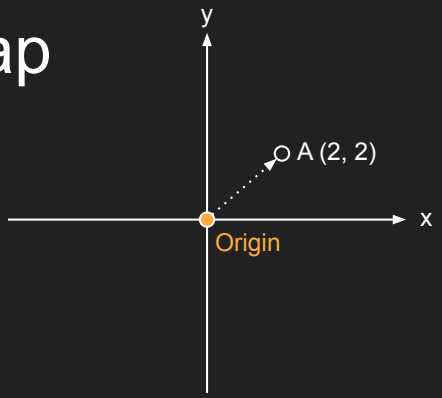
IT UNIVERSITY OF COPENHAGEN

Game Programming 25

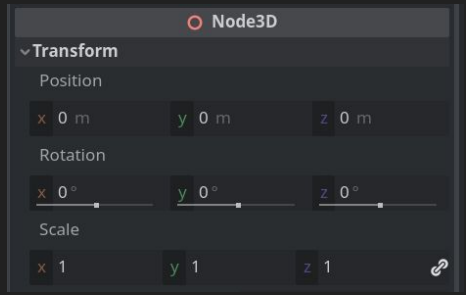
L04: Physics

Chris Carvelli

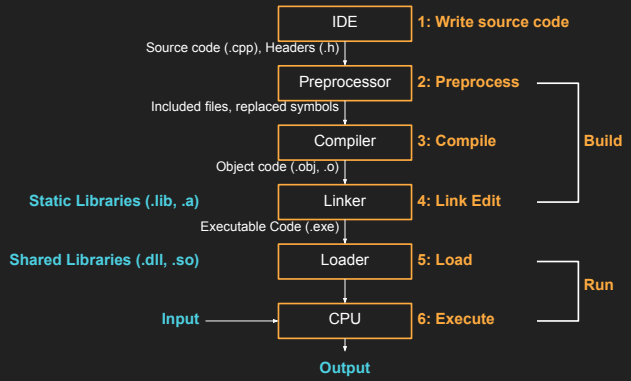
Recap



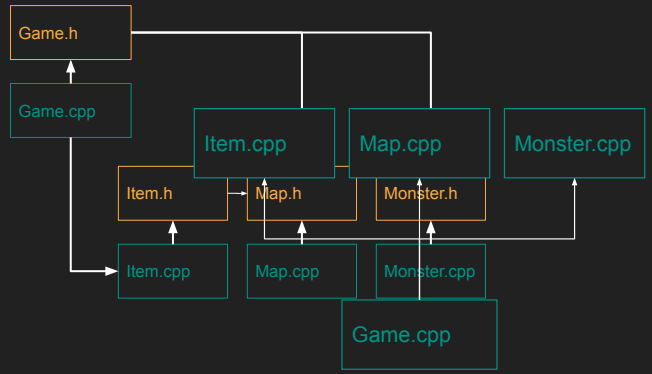
cartesian coordinates



transforms



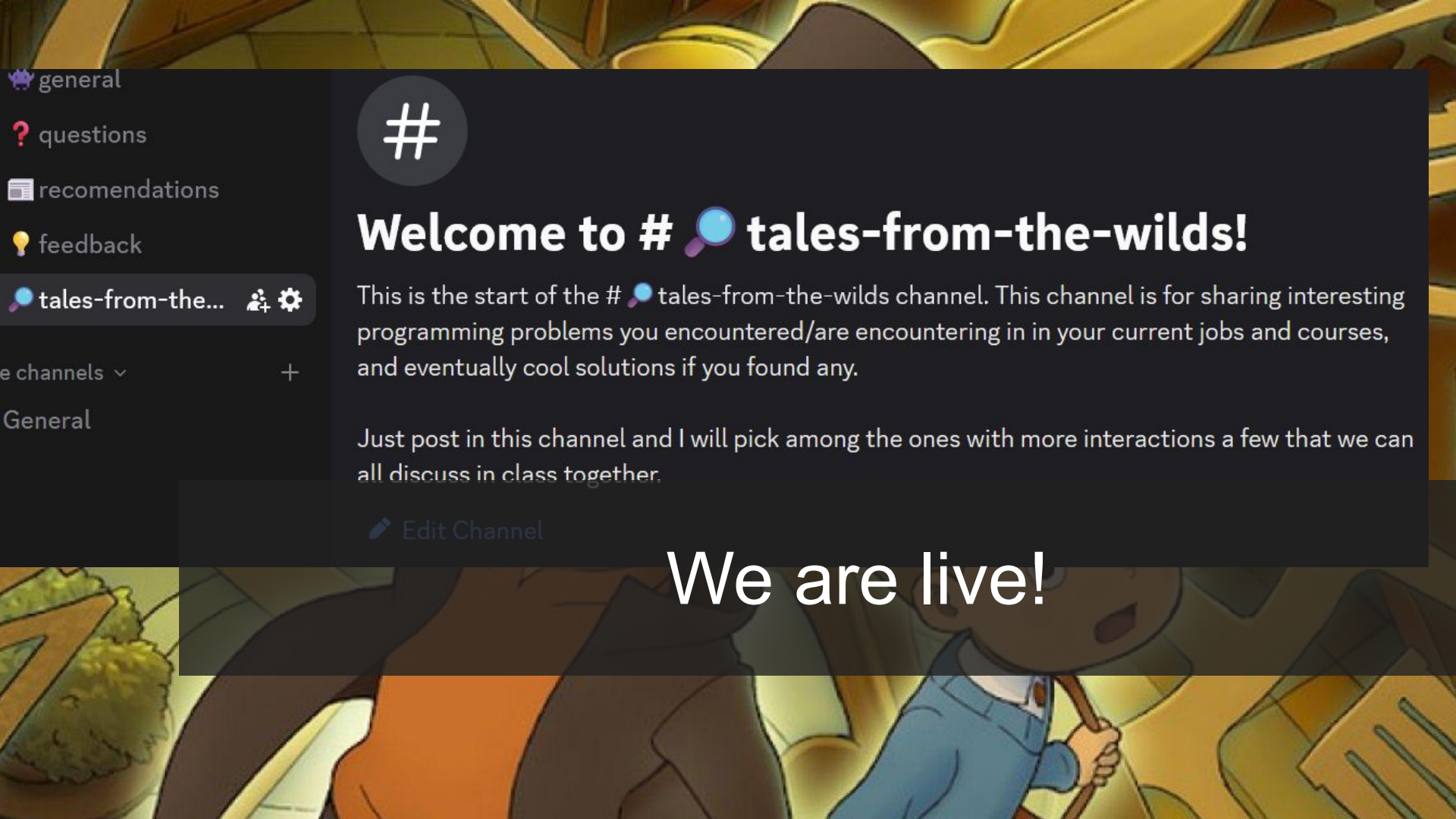
C++ building pipeline



C++ program structures

Agenda

1. Physics Simulation
2. Box2D


general
questions
recomendations
feedback
tales-from-the...
e channels
General
#
Welcome to # 🔎 tales-from-the-wilds!
This is the start of the # 🔎 tales-from-the-wilds channel. This channel is for sharing interesting programming problems you encountered/are encountering in in your current jobs and courses, and eventually cool solutions if you found any.
Just post in this channel and I will pick among the ones with more interactions a few that we can all discuss in class together.
Edit Channel
We are live!

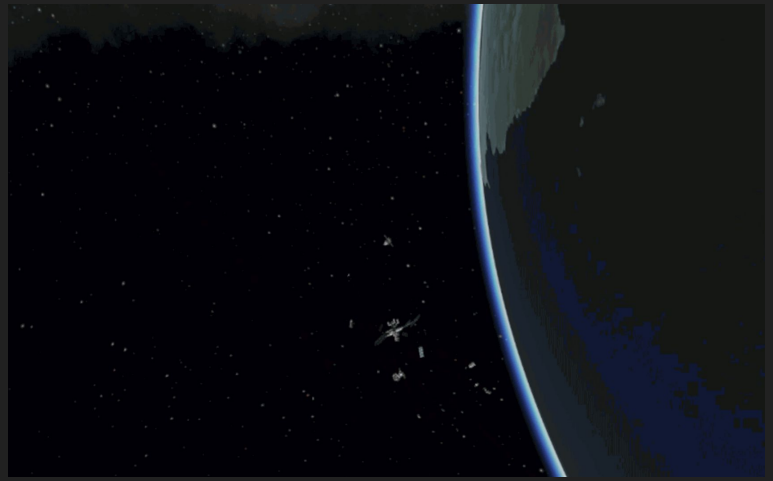
Early feedback results

(delayed again for technical issues…)

Exercise 03 review

L04 - Physics

Problem statement: How do we…



- …complement collision detection with realistic physics?
- …optimize physics simulations?

- …make physics simulation and our game communicate?
- …integrate 3rd party libraries in our project?

Agenda

1. Physics Simulation
2. Box2D

Physics concepts

- position
- speed — magnitude of the velocity
- velocity — change of position over time
- acceleration — change of velocity over time
- momentum — velocity scaled by mass

- force — vector quantity that causes a change of velocity

Rotational counterparts

- position
- speed
- velocity
- acceleration
- momentum

- force

- rotation
- angular speed
- rotational velocity
- rotational acceleration (rarely used)
- angular momentum

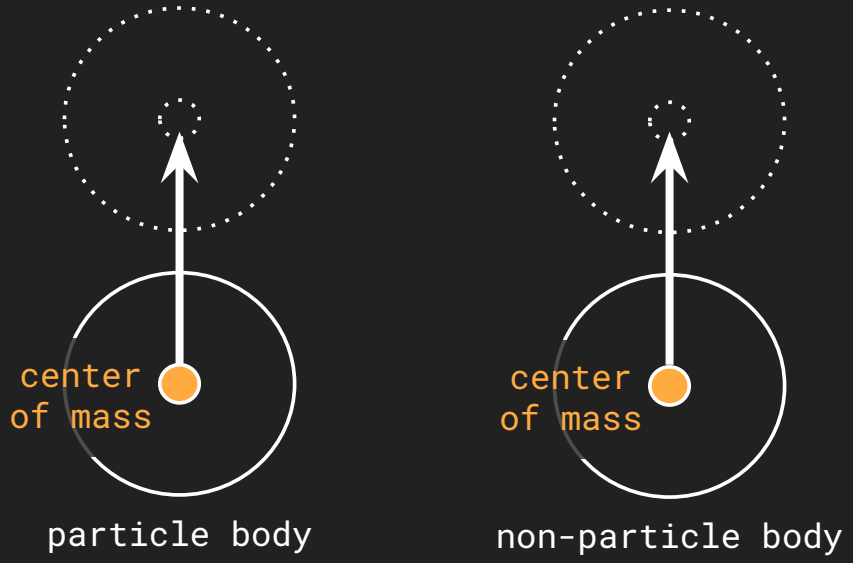
- torque

Non-particle bodies

So far, we did *very* simple physics simulations, mostly involving position, velocity and speed. Adding acceleration and forces is easy, same for their angular counterparts.

This works tho only because a silent assumption we made about our objects: they have no size and zero mass. They are de-facto particles!

This, however, is not how real world objects behave



Non-particle bodies

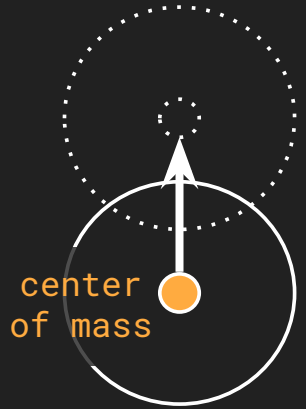
So far, we did *very* simple physics simulations, mostly involving position, velocity and speed. Adding acceleration and forces is easy, same for their angular counterparts.

This works tho only because a silent assumption we made about our objects: they have no size and zero mass. They are de-facto particles!

This, however, is not how real world objects behave

heavier objects have smaller reactions to the same forces

particle body
velocity = sum of forces



heavy body
velocity is scaled by mass

Non-particle bodies

So far, we did *very* simple physics simulations, mostly involving position, velocity and speed. Adding acceleration and forces is easy, same for their angular counterparts.

This works tho only because a silent assumption we made about our objects: they have no size and zero mass. They are de-facto particles!

This, however, is not how real world objects behave

particle body

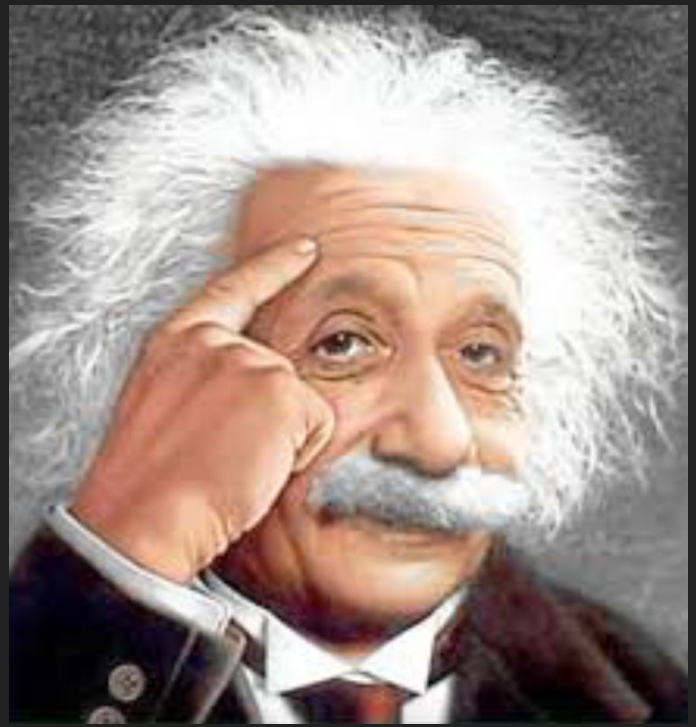
- forces can only be applied to center of mass
- entire force contributes to changing velocity

non-particle body

- forces can be applied anywhere
- off-center forces apply some energy to angular velocity

Other physics concepts

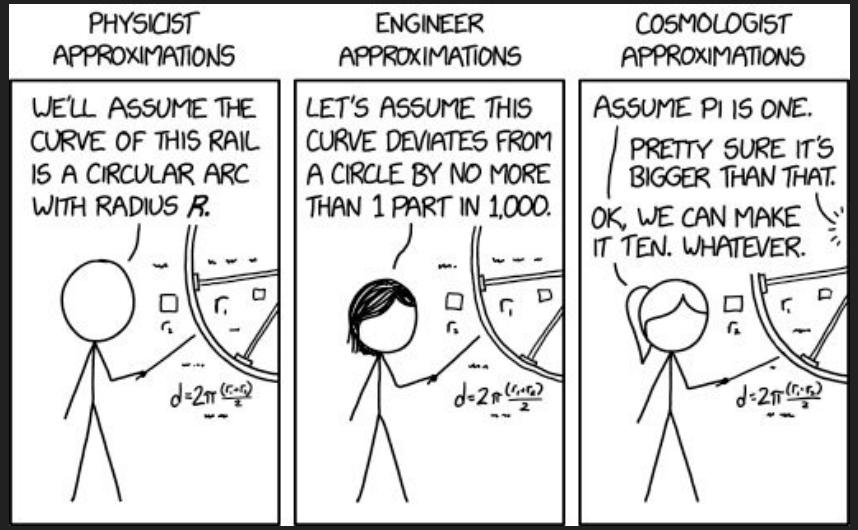
- Drag: Force that slow down an object
- AngularVelocity: Rotation
- Torque: Rotational force
- Mass: The weight of the object. Influences the effects of forces! (`Force = mass*acceleration`)



Realtime Physic Simulations

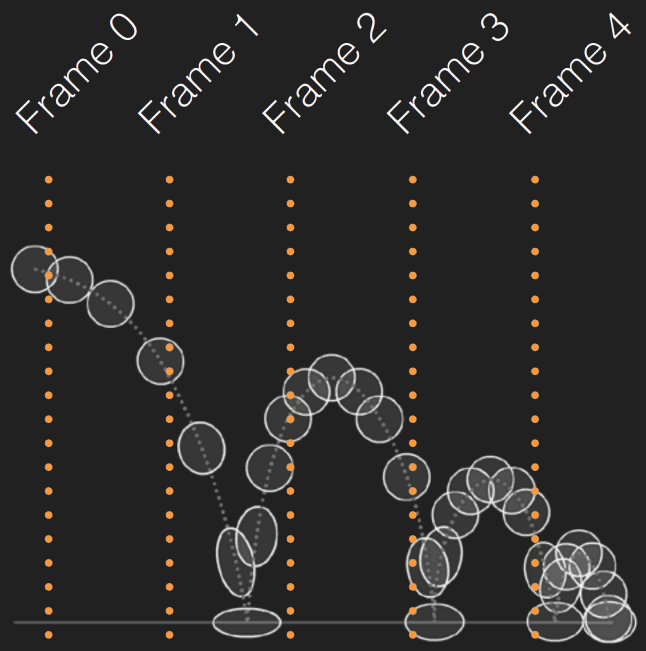
Physics simulation are expensive.

What tricks can we implement to make it less expensive?



Timing

- Physics simulation is usually performed in fixed steps (which makes the simulation more numerically stable)
- Steps
  - Fixed timestep
  - Number of physics simulations

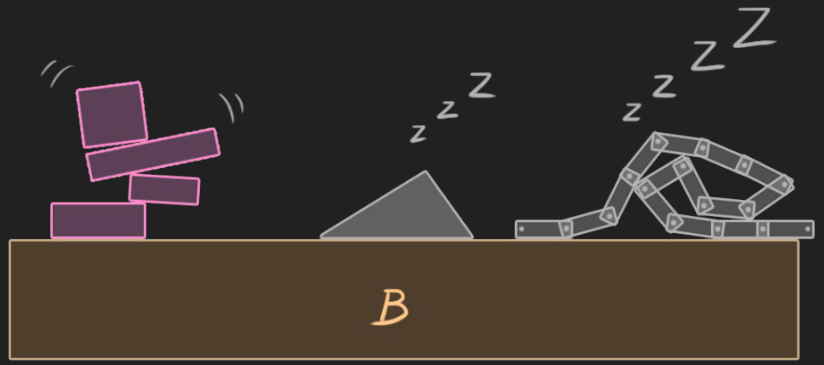


Types of physics bodies

- Static
  - No movement under simulation
  - Can be manually moved by user
  - Has no velocity
- Dynamic
  - Fully simulated
  - Mass = density * volume
  - Move according to forces
- Kinematic
  - Move according to velocity
  - Does not respond to forces

Sleep

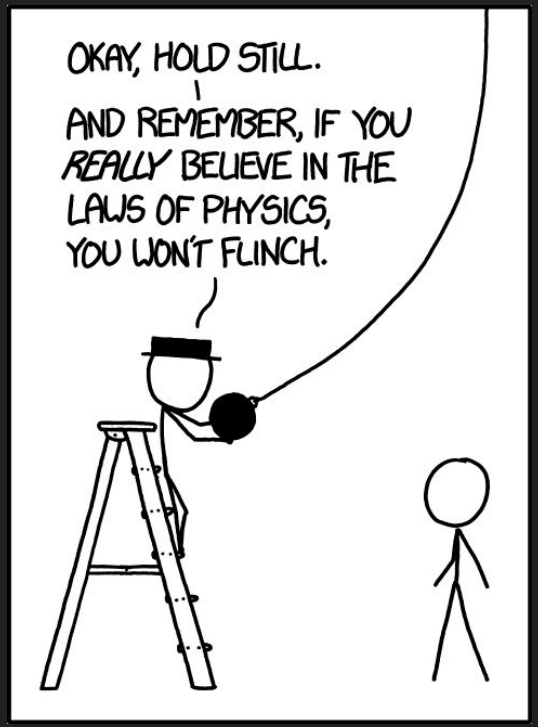
- A rigid body can be marked as sleeping
- Objects are put to sleep when velocity is under a given threshold
- Physics simulation are not performed on sleeping objects
- Objects are woken up on collisions and/or when force is applied.



Implementing physics concepts into

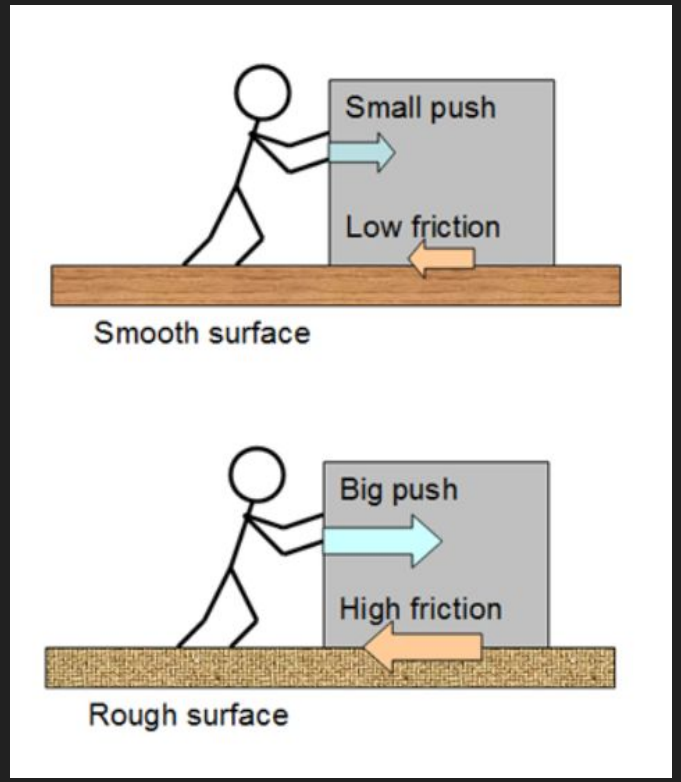
Physics is complex[Citation Needed], so we have to pick carefully what we want to implement in our physics system.

Velocity, angular velocity, mass and forces/torque are technically enough to simulate most physics phenomenon, but usually we add some “helper” concept to speed up our simulation, or to make it more stable.



Friction

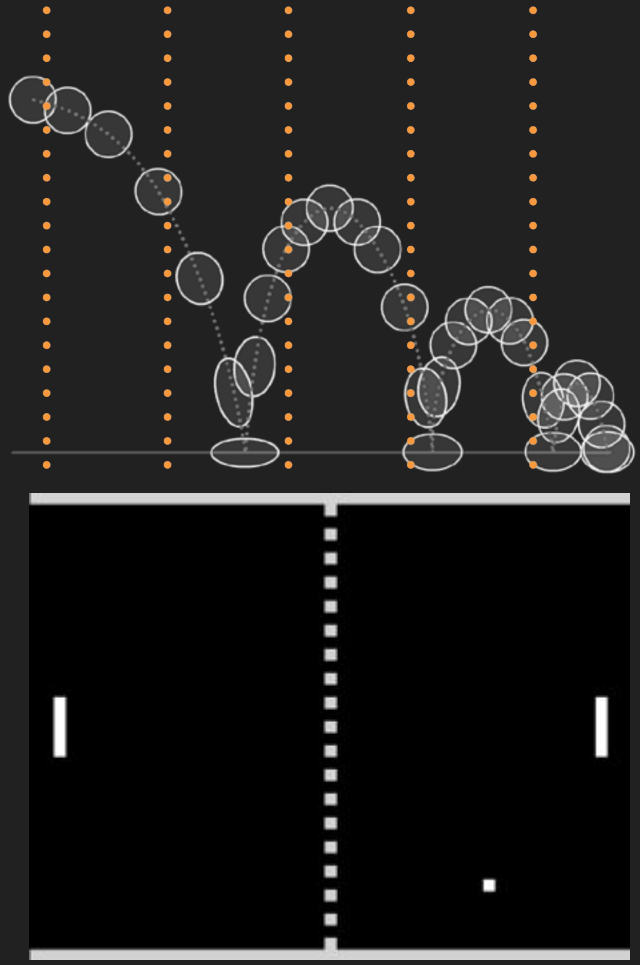
- Friction is the force resisting the relative motion of solid surfaces sliding against each other.
- Parameter of a body



Restitution / Elasticity

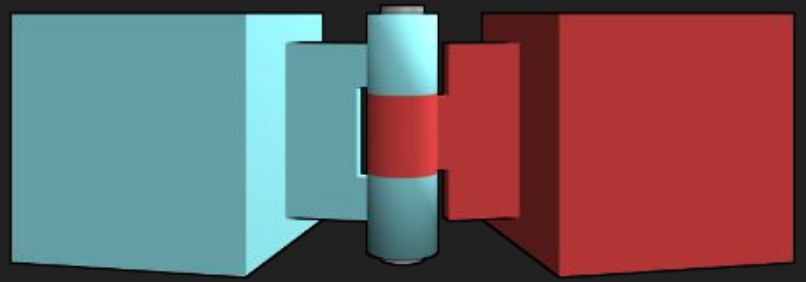
Physics material property

- Inelastic collision.
  - No bounce
  - Restitution=0
- Perfect elastic collision.
  - Full bounce
  - Restitution=1



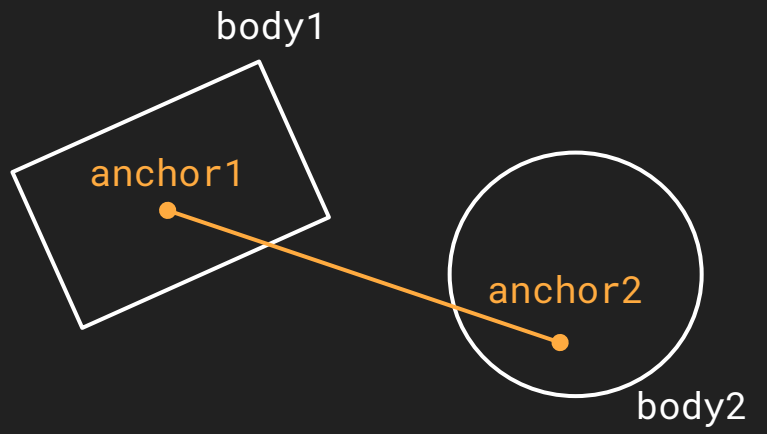
Constraint

- Often we want to limit the movement of an object
- Example:
  - A door can only be rotated around its hinge (in 3D)
  - Or fixed rotation (disallow rotation of object)
- These can be modelled by adding joints to objects (also used for rag doll effects)



Distance Joint

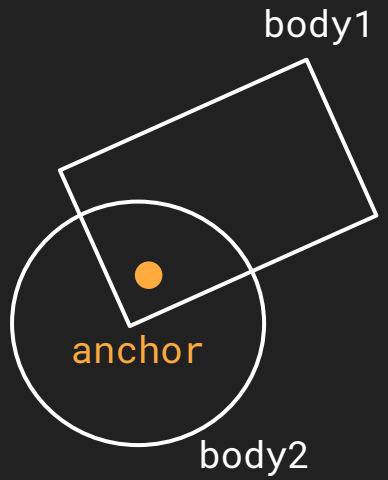
- Maintain a fixed distance between two anchor points on two rigid bodies
- Can be parametrized to achieve different effects, for example spring-like connections.




```
b2BodyId id_body_a, id_body_b;
b2DistanceJointDef joint_def;
joint_def.bodyIdA = id_body_a;
joint_def.bodyIdB = id_body_b;
joint_def.enableLimit = true;
joint_def.enableSpring = true;
// many more properties
b2JointId id_joint = b2CreateDistanceJoint(state->world_id, &joint_def);
```

Revolute Joint

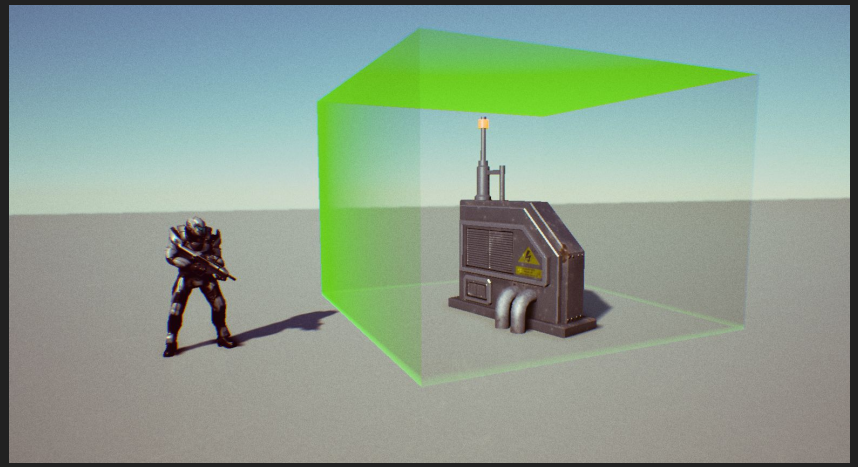
- Forces two bodies to share a common anchor point
- can be configured as motor or spring




```
b2BodyId id_body_a, id_body_b;
b2RevoluteJointDef joint_def;
joint_def.bodyIdA = id_body_a;
joint_def.bodyIdB = id_body_b;
joint_def.enableLimit = true;
joint_def.enableSpring = true;
// many more properties
b2JointId id_joint = b2CreateRevoluteJoint(state->world_id, &joint_def);
```

Sensors/Triggers

- Not really physics related, but very useful for gameplay
- A fixture in box2D can be set to be a sensor
- Other physics objects do not collide with sensors
- But collision callbacks are still invoked.



Common physics problems

- Physics engine cannot satisfy all constraints
- Numerical problems (e.g. oscillating objects)
- Incorrect world scale

Agenda

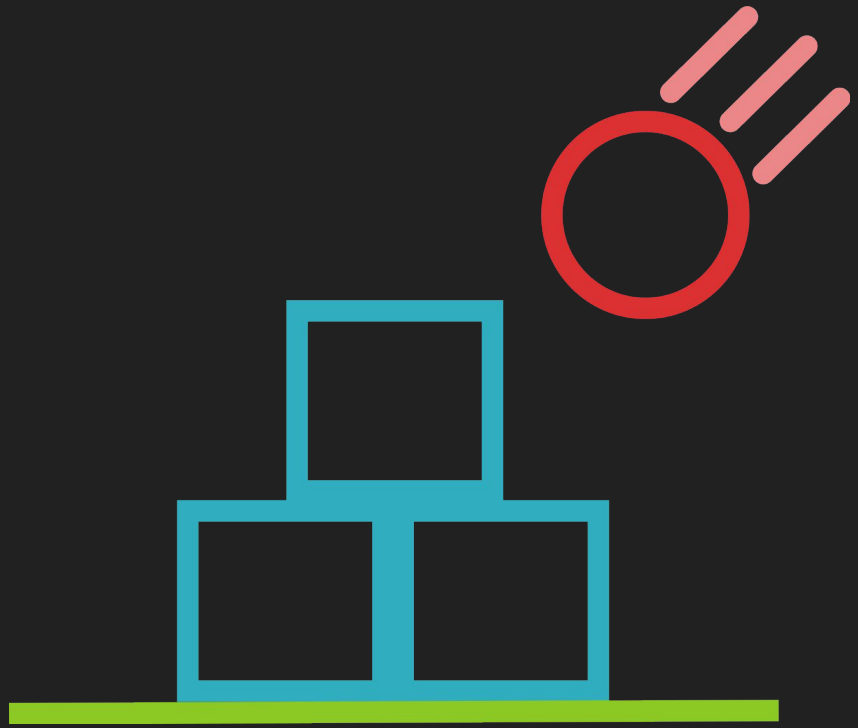
1. Physics Simulation
2. Box2D

Box2D

- C 2D Physics Engine
- Types begin with b2 prefix (e.g. b2World)
- https://box2d.org/
- https://box2d.org/documentation/

Important! We will use box2D version 3.1.0!

Be careful when searching online, many link/posts/discussions still refer to 2.X stuff

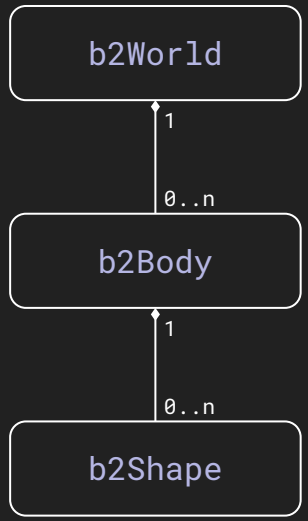


Important Facts

- Works with meters-kilogram-second (MKS) units
- Works well with moving shapes with size between 0.1 and 10 meters (ie: don’t work with windows coordinates)
- Using radians for angles

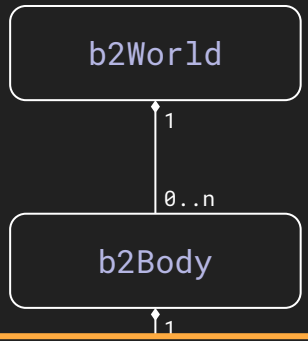
Core Entities

- b2World : collection of entities
  - gravity
  - sleeping on/off
  - continuous collisions on/off
- b2Body : non-deformable object(rigidbody)
  - type
  - transform
  - damping
- b2Shape : 2D geometrical object
  - physics material
  - density
  - collision filters
  - enable contact/hit events
  - sensor on/off



Core Entities

- b2World : collection of entities
  - gravity
  - sleeping on/off
  - continuous collisions on/off
- b2Body : non-deformable object(rigidbody)
  - type
  - transform
- IMPORTANT!
  box2d is slanted towards heavy physics simulations!
  Kinematic bodies are more performant than normal BUT they can collide with dynamic bodies only!
  If you’re in doubt, just make your body dynamic



Object lifecycle

All objects/entities in box2d follow the same lifecycle:

1. fill a descriptor data structure (`Def` prefix)
2. call `b2CreateXXX` with the descriptor structure
3. create function returns a `b2XXXId` identifier
4. destroy the object with `b2DestroyXXX`

```
b2WorldDef def_world = b2DefaultWorldDef();
// configure object through the `def` structure
b2WorldId id_world = b2CreateWorld(&def_world);

if(b2World_IsValid(id_world))
    b2DestroyWorld(id_world);
```

◆ b2CreateBody()

```
B2_API b2BodyId b2CreateBody ( b2WorldId        worldId,
                               const b2BodyDef * def )
```

Create a rigid body given a definition.

No reference to the definition is retained. So you can create the definition on the stack and pass it as a pointer.

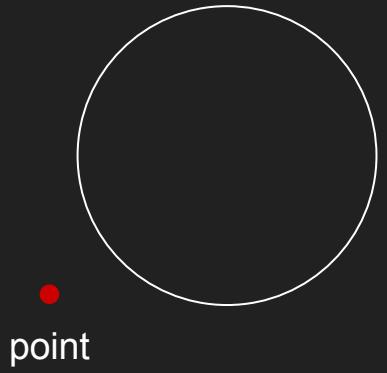
Creating a body

- by default, box2d will just perform simulation, you need to enable reporting of collision events
- Shapes are the colliders for the body (multiple shapes can e used for complex, possibly concave colliders)
- multiple types of shapes available (polygons, circles, capsules)
- NOTE: polygons have a hard limit to the number of vertices (default 8) for performance reasons

```
b2BodyDef body_def = b2DefaultBodyDef();
body_def.type = b2_dynamicBody;

b2ShapeDef shape_def = b2DefaultShapeDef();
shape_def.enableSensorEvents  = true;
shape_def.enableContactEvents = true;
shape_def.enableHitEvents     = true;

b2Polygon polygon = b2MakeBox(half_w, half_h);

b2BodyId id_body = b2CreateBody(world_id, &body_def);

b2ShapeId id_shape = b2CreatePolygonShape(
    id_body,
    &shape_def,
    &polygon
);
```

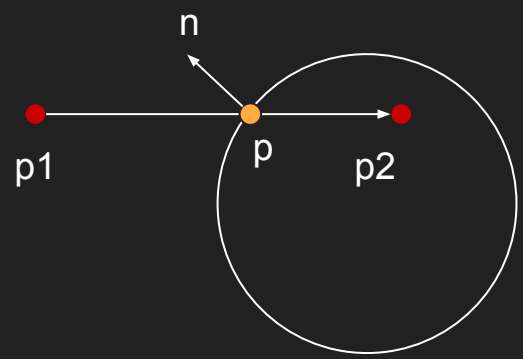
Geometric Queries - TestPoint

```
b2Vec2 point = { 0, 0 };
bool is_inside = b2Shape_TestPoint(id_shape, point);
```



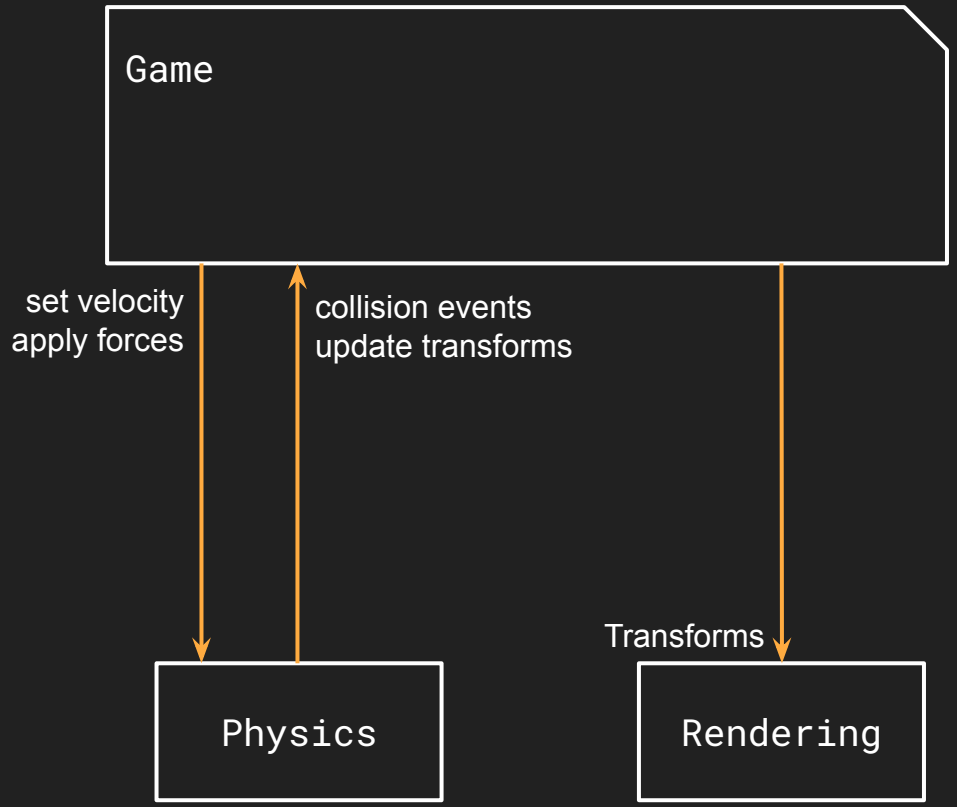
Geometric Queries - RayCast

```
b2Vec2 start = value_cast(b2Vec2, player->transform.position);
b2Vec2 translation;   // direction + max distance
b2QueryFilter filter; // same filtering logic as collisions
b2RayResult res = b2World_CastRayClosest(id_world, start,
translation, filter);
if(res.hit)
{
    b2Vec2 hit_pos = res.point;
    b2Vec2 hit_normal = res.normal;
    b2Vec2 hit_distance = start + translation * res.fraction;
    b2ShapeId hit_id_shape = res.shapeId;
    b2BodyId hit_id_body = b2Shape_GetBody(hit_id_shape);
}
```



Integration Box2D

- The physics world needs to be synchronized with the rest of the game, and vice versa
- Update physics world (impulses, forces, velocities)
- Update transforms
- Collision events



Integration

```
static void game_update(SDLContext* context, GameState* state)
{
    // read inputs and set velocities/apply forces accordingly

    b2World_Step(state->world_id, 1.0f / 60.0f, 4);

    // read updated transforms
    for(int i = 0; i < state->entities_alive_count; ++i)
    {
        Entity* entity = &state->entities[i];
        b2Vec2 physics_pos = b2Body_GetPosition(entity->body_id);
        b2Rot  physics_rot = b2Body_GetRotation(entity->body_id);
        entity->transform.position = value_cast(vec2f, physics_pos);
        entity->transform.rotation = b2Rot_GetAngle(physics_rot);
    }

    // ...
```

```
// cont.
// read ALL collision events (exercise shows how to do per-body)
b2ContactEvents events = b2World_GetContactEvents(state->world_id);

for(int i = 0; i < events.hitCount; ++i)
{
    b2ShapeId hit_id_shape_a = events.hitEvents[i].shapeIdA;
    b2ShapeId hit_id_shape_b = events.hitEvents[i].shapeIdB;
    b2Vec2 hit_normal = events.hitEvents[i].normal;
    b2Vec2 hit_point = events.hitEvents[i].point;
}

for(int i = 0; i < events.beginCount; ++i)
{
    b2ShapeId touch_id_shape_a = events.beginEvents[i].shapeIdA;
    b2ShapeId touch_id_shape_b = events.beginEvents[i].shapeIdB;
    b2Vec2 touch_normal = events.beginEvents[i].manifold.normal;
    int touch_points_count = events.beginEvents[i].manifold.pointCount;
    b2ManifoldPoint touch_points[2]; // more useful stuff in here
    for(int j = 0; j < touch_points_count; ++j)
        touch_points[j] = events.beginEvents[i].manifold.points[j];
}

for(int i = 0; i < events.endCount; ++i)
{
    // same as begin
}
```